KV Cache Engine
Interface Specification

LonghornSilicon LLM Inference Accelerator

Block 2 of 4 — KV Cache Compression / Decompression

Version: `kv-isa-0.2` (ChannelQuant codec)
Date: 2026-07-03
Status: Codec integrated; RTL signed off (all CI gates green, Sky130)
Author: Chaithu Talasila, The University of Texas at Austin
Reference: `LonghornSilicon/kv-cache-engine`

Scope. This document describes the externally-visible interface to the `kv_cache_engine` block as it will appear on the chip, on an FPGA prototype, or in the bit-accurate software model. A compiler backend targeting the KV cache engine writes against this interface; the hardware implementation must conform to it. **v0.2** replaces the retired TurboQuant+ codec (PolarQuant + QJL + Walsh–Hadamard rotation) with **ChannelQuant** — per-channel INT4 keys, per-token INT4 values, and static outlier-channel isolation — following the algorithm contract frozen in the sibling `channelquant` repository. This specification is stable for the KV cache engine block only and will be unified with the rest of the LonghornSilicon ISA when the Token Importance Unit and Memory Hierarchy Controller blocks land.

Contents

1. Block Overview

The KV cache engine manages on-chip storage of key and value tensors for transformer attention, with lossy compression to reduce DRAM bandwidth. It implements **ChannelQuant**, which quantizes on the axis that dominates the KV error budget for grouped-query-attention (GQA) models — a few fixed high-magnitude *key channels*:

- **Key path (per-channel INT4)**: keys are scaled *per channel* over a group of `G` tokens, then quantized to INT4. The top-k worst channels (tier CQ-4+) are held FP16 instead of INT4.
- **Value path (per-token INT4)**: values are scaled per token and quantized to INT4 (INT8 for the CQ-8 tier).

Tiers (`INFO_TIER`):

- **CQ-8**: per-token INT8 for both K and V.
- **CQ-4**: per-channel INT4 keys, per-token INT4 values.
- **CQ-4+**: CQ-4 plus k FP16 outlier key channels from a static calibrated ROM mask.

The compression flow is:

1. **Key group buffer**: accumulate `G` key tokens (`residual_buffer`).
2. **Per-channel scale**: take the per-channel max over the group (`amax_unit`, key mode) and freeze `D` per-channel FP16 scales (`scale_bank`).
3. **Quantize**: keep channels → INT4; outlier channels (CQ-4+) → FP16 (static ROM mask). Values quantize per token against a per-token FP16 scale.
4. **Store**: a unified per-channel SRAM record (§6).

Decompression is per channel: `code` · `scale` reconstructs each keep channel; outlier channels replay their stored FP16 directly. The read-side bus is **IEEE-754 fp32** (contract §1).

Compression: ≈4 effective bits/value, ≈3.8× vs. FP16, near-lossless.
Accuracy (HellaSwag acc_norm, n=1000): Qwen2-0.5B 0.421/0.415 (CQ-4/CQ-4+) vs. 0.426 FP16; Qwen2-1.5B 0.505/0.517 vs. 0.522 FP16 (CQ-4+ helps at D=128, not D=64).
Bit-exact reference: `sw/reference_model/channelquant_ref.{hpp,cpp}` and the frozen Python reference in `../channelquant/reference/`.

2. Address Space and Memory Map

The block exposes a 256-byte AXI-Lite slave register window. All registers are 32-bit, word-aligned. The base address is set at chip integration time.

Conventions. `RW` = read/write; `R` = read-only; `W1C` = write-1-to-clear; (syn) = value fixed at synthesis time. Writes to read-only registers are silently dropped. Reading reserved offsets returns `0xDEADBEEF`.

Table 1: AXI-Lite register window (offsets relative to base address), ISA v0.2.

Offset	Name	Access	Reset	Purpose
0x00	CTRL	RW	0x0	bit[0]: soft_reset (write-1 to pulse); bit[1]: enable
0x04	STATUS	R	0x1	bit[0]: idle; bit[3]: sram_full
0x08	INFO_DIM	R	(syn)	Head dimension VECTOR_DIM (D)
0x0C	INFO_TIER	R	(syn)	0 = CQ-8, 1 = CQ-4, 2 = CQ-4+
0x10	INFO_GROUP	R	(syn)	Key group size G (contract §3.1)
0x14	INFO_SRAM_DEPTH	R	(syn)	Number of SRAM entries
0x18	INFO_CR_K	R	(syn)	Key compression ratio, fixed-point 8.8
0x1C	INFO_CR_V	R	(syn)	Value compression ratio, fixed-point 8.8
0x20	INFO_VERSION	R	0x00020000	bits[31:24] major, [23:16] minor, [15:8] patch, [7:0] build
0x24	OCCUPANCY	R	0x0	Number of valid entries in SRAM
0x28	WRITE_ADDR	RW	0x0	Target write / key-group-base address
0x2C	READ_ADDR	RW	0x0	Target read address (write launches a decompress)
0x30	KV_SELECT	RW	0x0	bit[0]: 0 = key, 1 = value
0x34	IRQ_MASK	RW	0x0	bits[3:0]: interrupt enable mask
0x38	IRQ_STATUS	R/W1C	0x0	bits[3:0]: interrupt pending; write-1-to-clear
0x3C	INFO_OUTLIER_K	R	(syn)	Top-k FP16 outlier key channels (CQ-4+)
0x40	INFO_SCALE_DEPTH	R	(syn)	Per-channel scale-bank depth (= D)
0x44	INFO_RESID_DEPTH	R	(syn)	Residual-buffer depth (= G)

3. Streaming Data Interfaces

Two AXI-Stream interfaces carry the vector data. The AXI-Lite window in §2 is for control only.

3.1 s_axis_kv — Vector Write (Slave)

Signal	Width	Direction	Purpose
tdata	COORD_WIDTH (16)	in	One FP16 coordinate
tlast	1	in	Assert on the final coordinate of the token
tvalid	1	in	Handshake: data is valid
tready	1	out	Handshake: block is ready to accept
tuser	1	in	0 = key vector, 1 = value vector

Protocol:

- A complete token is exactly VECTOR_DIM beats. tlast asserts on the final beat.
- tuser selects the path: value tokens (and, in the CQ-8 tier, key tokens) compress per token; key tokens in CQ-4/CQ-4+ enter the *grouped* per-channel path.
- **Key grouping.** Program WRITE_ADDR to the group base, then stream G key tokens back-to-back. The engine buffers the group, freezes the per-channel scales, and emits the group's per-token records to base, base+1, ... (Partial-group flush, $g < G$, is a planned extension; the datapath already supports it.)

- Backpressure: `tready` deasserts during compression / SRAM write / group emit. Upstream must hold signals stable.

3.2 `m_axis_kv` — Vector Read (Master)

Signal	Width	Direction	Purpose
`tdata`	`OUT_WIDTH` (32)	out	One IEEE-754 fp32 decompressed coordinate
`tlast`	1	out	Asserted on the final coordinate
`tvalid`	1	out	Handshake: data is valid
`tready`	1	in	Handshake: downstream is ready to accept

Protocol. Writing `READ_ADDR` launches a decompress; the block then streams `VECTOR_DIM` fp32 beats, one channel per beat. The read bus is fp32 (contract §1); a downstream attention core may narrow to fp16 outside the codec's parity boundary.

3.3 Eviction Interface

Signal	Width	Direction	Purpose
`evict_needed`	1	out	Asserted when SRAM is full
`evict_addr`	$\lceil \log_2 $ `SRAM_DEPTH` $\rceil$	out	Address of entry to evict

This interface connects to the Memory Hierarchy Controller (block 4).

4. Logical Operations

Operation	Inputs	Outputs	Description
`KV_QUERY`	—	INFO struct	Read all `INFO_*` registers
`KV_RESET`	—	—	Soft reset: clear SRAM, reset FSM
`KV_ENABLE`	—	—	Set bit[1] of `CTRL`
`KV_COMPRESS_KEY`	base, $G \times D$ coords	—	Buffer + compress a key *group*; store G per-token records
`KV_COMPRESS_VALUE`	addr, D coords	—	Compress and store a value token at `addr`
`KV_DECOMPRESS_KEY`	addr	D fp32 coords	Read and decompress a key token from `addr`
`KV_DECOMPRESS_VALUE`	addr	D fp32 coords	Read and decompress a value token from `addr`
`KV_STATUS`	—	STATUS bits	Read the `STATUS` register
`KV_OCCUPANCY`	—	count	Read the `OCCUPANCY` register

4.1 Worked Lowering Example

Stage 1: setup. Query hardware configuration once per compilation:

```
info = KV_QUERY()   // -> dim=64, tier=1(CQ-4), group=128, outlier_k=0,
   sram_depth=16
KV_RESET(); KV_ENABLE()
```

Stage 2: store phase. Values are per-token; keys are grouped:

```
for token_idx in new_tokens:                          // values: per token
    KV_COMPRESS_VALUE(token_idx % info.sram_depth, v_vectors[token_idx])

for group in key_groups_of(new_tokens, info.group): // keys: per G-token
    group
    KV_COMPRESS_KEY(group.base_addr, group.k_vectors)   // stream G*D
       coords
```

Stage 3: attention phase. Decompress cached K/V for attention:

```
for cached_idx in range(num_cached):
    k_hat = KV_DECOMPRESS_KEY(cached_idx)   // D fp32 coords
    v_hat = KV_DECOMPRESS_VALUE(cached_idx)
    score = dot(query, k_hat) / sqrt(D)
    decision = PC_PUSH_TILE(score)          // block 1: precision
       controller
    out += (fp16_matmul if decision == FP16 else int8_matmul)(softmax(
       score), v_hat)
```

4.2 Compiler Binding Patterns

MLIR dialect. Define `lhsi.kv.*` ops mirroring §4; the lowering pass rewrites attention ops to emit `lhsi.kv.compress_key` (grouped), `lhsi.kv.decompress_key`, etc. **TVM Relax / TIR**, **ONNX CustomOp**, and **custom IR** backends bind analogously to the C API in §5.

5. C API Stub

```
/* lhsi_kv_cache_engine.h --- LonghornSilicon KV Cache Engine driver API
    (v0.2). */

#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>

typedef struct {
    uint32_t vector_dim;       /* head dim D
                                  */
    uint32_t tier;             /* 0=CQ-8, 1=CQ-4, 2=CQ-4+
                               */
    uint32_t key_group;        /* G: tokens per per-channel key group
                */
    uint32_t outlier_k;        /* top-k FP16 outlier key channels (CQ
       -4+)     */
    uint32_t scale_depth;      /* per-channel scale-bank depth (= D)
                */
    uint32_t resid_depth;      /* residual-buffer depth (= G)
                          */
    uint32_t sram_depth;
    uint32_t cr_k_fixed;       /* compression ratio K, fixed-point 8.8
                */
```

```
    uint32_t cr_v_fixed;       /* compression ratio V, fixed-point 8.8
                */
    uint32_t version;          /* 0x00020000 = v0.2
                  */
} lhsi_kv_info_t;

typedef struct lhsi_kv_handle lhsi_kv_handle_t;

int  lhsi_kv_open (const char *device, lhsi_kv_handle_t **out);
void lhsi_kv_close(lhsi_kv_handle_t *h);
int  lhsi_kv_query(lhsi_kv_handle_t *h, lhsi_kv_info_t *out);
int  lhsi_kv_reset (lhsi_kv_handle_t *h);
int  lhsi_kv_enable(lhsi_kv_handle_t *h, bool enable);

/* Compress + store a KEY GROUP of `g` tokens (g <= key_group) at
   base_addr.
 * coords is g*dim FP16 values, token-major. */
int  lhsi_kv_compress_key_group(lhsi_kv_handle_t *h, uint32_t base_addr,
                                const uint16_t *coords_f16, size_t g,
                                   size_t dim);

/* Compress + store a single VALUE token at addr (dim FP16 values). */
int  lhsi_kv_compress_value(lhsi_kv_handle_t *h, uint32_t addr,
                            const uint16_t *coords_f16, size_t dim);

/* Read + decompress a key / value token: dim fp32 values out. */
int  lhsi_kv_decompress_key  (lhsi_kv_handle_t *h, uint32_t addr,
                              float *coords_f32, size_t dim);
int  lhsi_kv_decompress_value(lhsi_kv_handle_t *h, uint32_t addr,
                              float *coords_f32, size_t dim);

typedef struct { bool idle; bool sram_full; uint32_t occupancy; }
   lhsi_kv_status_t;
int  lhsi_kv_status(lhsi_kv_handle_t *h, lhsi_kv_status_t *out);
```

Return codes follow the standard convention: `0` on success, `-errno` on failure.

6. Compression Algorithm: ChannelQuant

ChannelQuant is a per-channel/per-token integer quantization scheme designed for KV-cache compression in GQA transformer inference. It follows the KIVI/KVQuant recipe; the algorithm contract, reference model, and golden vectors are frozen in the sibling `channelquant` repository (contract v0.2). Rounding is round-half-to-even; INT4 clamps to $[-8, 7]$, INT8 to $[-128, 127]$; scales are FP16 with EPS $= 2^{-14}$.

6.1 Key Path — Per-Channel INT4

Keys are quantized on the *channel* axis, because the KV error on GQA models is dominated by a few fixed high-magnitude key channels.

1. Buffer a group of `G` key tokens.

2. For each channel c, take $a_c = \max_t |k_{t,c}|$ over the group and form the FP16 scale $s_c = \max(a_c/q_{\max}, \text{EPS})$ with $q_{\max} = 7$.
3. Quantize each keep channel: $\hat{q}_{t,c} = \text{clamp}(\text{round}(k_{t,c}/s_c))$ to INT4.
4. **CQ-4+ outlier lane**: the top-k channels from the static calibrated ROM mask (`../channelquant/reference/masks/`, $k = 2$) are stored FP16 rather than INT4.

6.2 Value Path — Per-Token INT4

Each value token is scaled by $s_t = \max(\max_c |v_{t,c}|/q_{\max}, \text{EPS})$ and quantized to INT4 (INT8 with $q_{\max} = 127$ for the CQ-8 tier). No grouping.

6.3 Unified Per-Channel SRAM Record

Each stored key token is a per-channel record {tag, $D \times$ FP16 field, $D \times$ INT4 code}:

- **keep channel** → {group scale s_c, INT4 code};
- **outlier channel** → {raw FP16 value, code +1}, so that decompress code · field widens the FP16 exactly — no separate sidecar region and no read-side mask.

Decompression is uniform per channel: $\hat{x}_c = \text{code}_c \cdot \text{field}_c$ in fp32. Value records store {FP16 scale, packed INT4/INT8 payload} and dequantize the same way.

6.4 Effective Bits and Compression

Path	Per-value bits	Notes
Key (CQ-4)	$4 + 16/G$	INT4 code + amortized per-channel scale
Key (CQ-4+)	$4 + 16/G + \frac{k}{D}(16-4)$	plus the FP16 outlier lane
Value (INT4)	$4 + 16/D$	INT4 code + per-token scale

At D=64, G=128, k=2: mean ≈4.2 bits/value, ≈3.8× vs. FP16's 16 bits.

6.5 Serialized Datapath

Each compute core (scale / quant / dequant) carries an FP16 divider. Rather than D parallel units (a single wide combinational divide cone that stalls place-and-route), the datapath **serializes one shared unit** walked across the D channels, bit-exact with the behavioral oracle. This is what lets the block synthesize, pass formal RTL≡netlist equivalence, and place-and-route on Sky130 within the CI time budgets.

7. Synthesis-Time Configuration

Gate proxy. The synthesis / formal / OpenLane CI gates run a deliberately small default proxy (the SRAM and residual buffer are behavioral flip-flops, no Sky130 macro); the shipped D/G/depth are set per instantiation.

Parameter	Default	Range	Notes
VECTOR_DIM	64	16, 32, 64, 128	Head dimension D
TIER	1	0, 1, 2	0=CQ-8, 1=CQ-4, 2=CQ-4+
KEY_GROUP	128	≥ 2	Tokens per per-channel key group (G)
OUTLIER_K	0	0, 2	FP16 outlier key channels (CQ-4+)
SCALE_WIDTH	16	16	FP16 scale width
COORD_WIDTH	16	16	FP16 input coordinate width
OUT_WIDTH	32	32	fp32 decompressed output width
SRAM_DEPTH	16	2–4096	Behavioral reg array for Sky130; real macros at 16FFC

8. Bit-Accurate Reference

Three implementations are bit-exact with each other (3-way parity, contract §8):

1. **Python**: `../channelquant/reference/channelquant_ref.py` — the frozen algorithm reference + golden vectors.
2. **C++**: `sw/reference_model/channelquant_ref.{hpp,cpp}` — a 1:1 port of the RTL behavioral cores.
3. **SystemVerilog**: `rtl/kv_cache_engine.sv` + `cq_key_path.sv` / `cq_value_path.sv` / `cq_units_syn.sv` + support — synthesizable RTL.

The RTL is verified bit-exact against the golden vectors by `makesim_cq` / `sim_kpath` / `sim_vpath` / `sim_top`, and the C++/Python legs by `make-Csw/reference_modeltest-all`.

9. Integration Phases

Phase	Timeline	Compiler targets	We provide
0	now	Bit-accurate Python and C++ reference models	Models + ISA + tests + Sky130 sign-off
1	when FPGA arrives	AXI-Lite + AXI-Stream on Zynq UltraScale+	Vivado bitstream + driver
2	after all 4 blocks complete	Multi-block FPGA project	Integrated attention pipeline
3	post-tape-out (2027+)	PCIe-attached custom chip	TSMC 16FFC silicon + Linux driver

10. Interaction with Other Blocks

- **Block 1 (Precision Controller / ACU)**: receives the fp32 decompressed K/V from this block for attention; it decides INT8 vs FP16 routing per tile from the scores. The controller is codec-agnostic — it never sees the KV storage format — so the ChannelQuant codec change requires no ACU change.

- **Block 3 (Token Importance Unit)**: provides token-importance scores informing eviction when SRAM is full.
- **Block 4 (Memory Hierarchy Controller)**: receives `evict_needed` / `evict_addr` when SRAM is full.

11. Open Questions

1. Partial-group flush ($g < G$): the datapath supports it; the top's stream framing currently auto-flushes at full G only. Which sideband signals a partial group (control-register bit vs. stream marker)?
2. Should the outlier count k and the ROM mask be run-time reloadable (per deployed model) rather than fixed at synthesis?
3. Is a per-token DWB-style tier selector (2/4/8-bit) worth the controller cost, given per-channel INT4 already recovers ∼FP16?
4. Are the fp16 scale / fp32 read-bus widths sufficient for larger heads ($D = 256$)?
5. Should the eviction policy be configurable via registers (LRU, FIFO, importance-based)?

12. Change Log

- `kv-isa-0.2` (2026-07-03): Codec replaced TurboQuant+ → ChannelQuant (per-channel INT4 keys, per-token INT4 values, CQ-4+ outlier lane). New `INFO_TIER`/`GROUP`/`OUTLIER_K`/`SCALE_DEPTH`/`RESID_DEPTH` registers; `INFO_PQ_BITS` / `INFO_QJL_BITS` removed. fp32 read bus. RTL fully integrated and signed off (all CI gates green, Sky130).
- `kv-isa-0.1` (2026-05-14): First public draft (TurboQuant+ codec, retired).